1 Implemented spectral calibration model

1.1 Indices, data, and campaign preprocessing

Let $s \in \{1, \dots, M\}$ index sensors, let $k \in \{1, \dots, K\}$ index aligned calibration experiments, and let $j \in \{1, \dots, J\}$ index the discrete frequency bins of the common calibration band. After preprocessing, the calibration data are represented by an observation tensor

$$Y_{s,k,j} > 0, \qquad (s, k, j) \in \{1, \dots, M\} \times \{1, \dots, K\} \times \{1, \dots, J\},$$

in linear power units.

Before alignment, sensor s provides a timestamped series

$$\{(\tau_{s,r}, x_{s,r,1}, \dots, x_{s,r,J})\}_{r=1}^{R_s},$$

where $\tau_{s,r}$ is the record timestamp and the $x_{s,r,j}$ are PSD values in dB.

The implemented campaign preprocessing stage does the following.

1. It loads one campaign directory from the `data/campaigns/` root and parses one CSV per sensor.
2. It sorts each sensor series by timestamp and validates that every retained sensor already uses the same PSD frequency grid.
3. It chooses the shortest sensor series as the alignment anchor, so every retained realization is observable across the full sensor set.
4. It aligns records greedily by timestamp: for each anchor timestamp, every other sensor contributes the nearest later record within an alignment tolerance, while preserving time order and dropping unmatched rows.
5. It converts the aligned PSD tensor from dB to linear power, producing $Y_{s,k,j}$.
6. It computes ranking diagnostics on the aligned dB tensor and chooses a reliable sensor $s_\star$ as the highest record-wise correlation winner that is *not* flagged as a campaign-wide PSD-distribution outlier; if every retained sensor is an outlier, it falls back to the record-wise winner.
7. It sets the nominal gain baseline to unity for every sensor and frequency bin.

The alignment tolerance is either supplied explicitly by the caller or inferred from the median campaign sampling period and then clipped to a conservative range. The aligned experiment timestamp stored by the adapter is the median timestamp across sensors for each retained realization.

Denote the resulting nominal gain baseline by $G^{(0)}_{s,j} > 0$. In the active campaign workflow the baseline is

$$G^{(0)}_{s,j} = 1, \qquad s = 1, \ldots, M,\ j = 1, \ldots, J. \tag{1}$$

So the learned calibration is a *relative* network calibration around a unit reference scale. Unlike older versions of the project, the current repository does not ingest external per-sensor nominal response CSV files.

1.2 Observation model

The fitted calibration model is

$$Y_{s,k,j} = G_{s,j}\, S_{k,j} + N_{s,j} + \varepsilon_{s,k,j}, \tag{2}$$

where:

- $S_{k,j} \geq 0$ is the latent common spectrum at experiment k and bin j;
- $G_{s,j} > 0$ is the multiplicative gain of sensor s;
- $N_{s,j} > 0$ is the additive sensor floor;
- $\varepsilon_{s,k,j}$ is a residual term.

Conditionally on the current latent spectra, the working likelihood is Gaussian:

$$\varepsilon_{s,k,j} \mid S_{k,j} \sim \mathcal{N}(0, \sigma^2_{s,j}), \tag{3}$$

with sensor- and frequency-dependent residual variance $\sigma^2_{s,j} > 0$.

1.3 Reparameterization used by the implementation

The code does not optimize directly in $(G_{s,j}, N_{s,j})$. Instead it uses the parameterization

$$G_{s,j} = G^{(0)}_{s,j} \exp(\gamma_{s,j}), \tag{4}$$

where $\gamma_{s,j} \in \mathbb{R}$ is the residual *log-gain correction*, and

$$N_{s,j} = \phi(\eta_{s,j}), \qquad \phi(\eta) = \log(1 + e^{\eta}), \tag{5}$$

where $\eta_{s,j} \in \mathbb{R}$ is an unconstrained *noise parameter*. Thus positivity of the additive floor is enforced by the softplus map (5), not by post-hoc clipping.

The derivative of (5) is

$$\phi'(\eta) = \frac{1}{1 + e^{-\eta}}, \tag{6}$$

which is used in the Gauss–Newton linearization.

1.4 Identifiability

The model is invariant under

$$G_{s,j} \mapsto c_j G_{s,j}, \qquad S_{k,j} \mapsto c_j^{-1} S_{k,j}, \qquad c_j > 0,$$

so it is identifiable only up to a frequency-wise multiplicative scale.

The implementation resolves this by projecting the residual log-gains to zero mean across sensors:

$$\frac{1}{M} \sum_{s=1}^{M} \gamma_{s,j} = 0, \qquad j = 1, \ldots, J. \tag{7}$$

Equivalently,

$$\frac{1}{M} \sum_{s=1}^{M} \log G_{s,j} = \frac{1}{M} \sum_{s=1}^{M} \log G_{s,j}^{(0)} + \frac{1}{M} \sum_{s=1}^{M} \gamma_{s,j} = 0, \tag{8}$$

because the active campaign workflow uses the unity baseline (1). More generally, the core solver would only require the baseline gains to have zero-mean log scale across sensors.

Under (7), the latent spectra $\{S_{k,j}\}$ live on the common network reference scale.

1.5 Reliable-sensor prior

Let $s_\star$ denote the sensor believed to be the most reliable one. In the current campaign workflow, this sensor is not chosen manually. Instead the code first ranks sensors by record-wise cumulative cross-correlation, then filters those candidates against a campaign-wide PSD-distribution similarity diagnostic. The reliable sensor is the first record-wise winner that is not flagged as a distribution-shape outlier; if all retained sensors are flagged, the code uses the record-wise winner anyway.

Once $s_\star$ has been selected, the implementation does *not* impose a hard reference constraint. Instead it uses two soft mechanisms:

1. A quadratic anchor on the residual log-gain of the reliable sensor, favoring
$$\gamma_{s_\star,j} \approx 0,$$
meaning that the reliable sensor should stay close to the unit network reference gain, or more generally to the nominal baseline supplied to the core solver.

2. An optional multiplicative boost of its inverse-variance weight in the latent-spectrum update.

This keeps the reliable sensor influential without forcing all mismatch into the rest of the network.

1.6 Initialization

Let $\mathcal{K}_{\text{train}} \subseteq \{1, \dots, K\}$ be the training experiments used for fitting. The implementation initializes:

$$\gamma_{s,j}^{(0)} = 0, \tag{9}$$
$$G_{s,j}^{(0,\text{fit})} = G_{s,j}^{(0)}, \tag{10}$$
$$N_{s,j}^{(0)} = 0.1\, Q_{0.05}(\{Y_{s,k,j} : k \in \mathcal{K}_{\text{train}}\}), \tag{11}$$
$$\eta_{s,j}^{(0)} = \phi^{-1}(N_{s,j}^{(0)}), \tag{12}$$
$$S_{k,j}^{(0)} = \text{median}_s \left[\frac{Y_{s,k,j} - N_{s,j}^{(0)}}{G_{s,j}^{(0,\text{fit})}} \right]_+, \tag{13}$$

where $[\cdot]_+$ denotes clipping to a small positive floor for numerical stability.

The initial residual variance is

$$\sigma_{s,j}^{2(0)} = \max\left(\frac{1}{|\mathcal{K}_{\text{train}}|} \sum_{k \in \mathcal{K}_{\text{train}}} (Y_{s,k,j} - G_{s,j}^{(0,\text{fit})} S_{k,j}^{(0)} - N_{s,j}^{(0)})^2,\ \sigma_{\min}^2 \right). \tag{14}$$

1.7 Second-difference smoothers

Let $f_1 < \cdots < f_J$ denote the actual frequency grid used by the aligned campaign tensor. The implementation does *not* assume exact unit spacing. Instead it normalizes the grid by the median spacing

$$\tilde{h} = \text{median}(f_{j+1} - f_j), \qquad j = 1, \dots, J-1,$$

and defines normalized coordinates

$$x_1 = 0, \qquad x_j = \sum_{u=1}^{j-1} \frac{f_{u+1} - f_u}{\tilde{h}}, \qquad j = 2, \ldots, J.$$

For row $r \in \{1, \ldots, J-2\}$, let

$$h_r^- = x_{r+1} - x_r, \qquad h_r^+ = x_{r+2} - x_{r+1}.$$

The implemented curvature operator uses the three-point non-uniform second difference

$$(D_2 x)_r = \frac{2}{h_r^-(h_r^- + h_r^+)} x_r - \frac{2}{h_r^- h_r^+} x_{r+1} + \frac{2}{h_r^+(h_r^- + h_r^+)} x_{r+2}.$$

The smoother then uses the corresponding Gram matrix

$$L = D_2^\top D_2. \tag{15}$$

Crucially, the gain smoother acts on $\gamma_s = (\gamma_{s,1}, \ldots, \gamma_{s,J})^\top$, i.e. on *log-gain correction*, and the noise smoother acts on $\eta_s = (\eta_{s,1}, \ldots, \eta_{s,J})^\top$, i.e. on the *softplus preimage*, not directly on N_s.

On an exactly uniform grid this reduces, up to the common scaling induced by the normalized coordinates, to the usual second-difference smoother.

1.8 Alternating estimation actually implemented

The estimator runs for a fixed number of alternating iterations $t = 0, \ldots, T-1$.

Step 1: inverse-variance weights

At iteration t, define

$$w_{s,j}^{(t)} = \frac{1}{\max(\sigma_{s,j}^{2(t)}, \sigma_{\min}^2)}. \tag{16}$$

If a reliable-weight boost $\beta_{\text{rel}} > 0$ is used, then

$$w_{s_\star,j}^{(t)} \leftarrow \beta_{\text{rel}}\, w_{s_\star,j}^{(t)}. \tag{17}$$

Step 2: latent-spectrum update

Given current gains and additive floors, the latent spectrum is updated by weighted least squares:

$$S_{k,j}^{(t+1)} = \left[\frac{\sum_{s=1}^{M} w_{s,j}^{(t)} G_{s,j}^{(t)} \left(Y_{s,k,j} - N_{s,j}^{(t)}\right)}{\sum_{s=1}^{M} w_{s,j}^{(t)} \left(G_{s,j}^{(t)}\right)^2} \right]_+ . \tag{18}$$

Step 3: frequency-only low-information diagnostic

The implementation measures how much the latent spectrum varies across calibration experiments:

$$v_j^{(t+1)} = \mathrm{Var}_{k\in\mathcal{K}_{\mathrm{train}}}(S_{k,j}^{(t+1)}). \tag{19}$$

Let

$$\mathcal{J}_+ = \{j : v_j^{(t+1)} > \varepsilon\}.$$

If $\mathcal{J}_+ = \varnothing$, then every frequency is declared low-information and

$$\omega_j^{\mathrm{freq}} = \omega_{\min}, \qquad m_j^{\mathrm{freq}} = 1, \tag{20}$$

for all j.

Otherwise define the threshold

$$\tau_v = \max\Big(\rho_v \ \mathrm{median}\{v_j^{(t+1)} : j \in \mathcal{J}_+\},\ \varepsilon\Big), \tag{21}$$

where $\rho_v > 0$ is the configured threshold ratio. Then

$$\omega_j^{\mathrm{freq}} = \mathrm{clip}\left(\frac{v_j^{(t+1)}}{\tau_v},\ \omega_{\min},\ 1\right), \qquad m_j^{\mathrm{freq}} = \mathbf{1}\{v_j^{(t+1)} < \tau_v\}. \tag{22}$$

Step 4: sensor-specific information weights

The frequency-only diagnostic is then refined into a sensor-specific information weight. Let

$$\bar{S}_j^{(t+1)} = \frac{1}{|\mathcal{K}_{\mathrm{train}}|} \sum_{k\in\mathcal{K}_{\mathrm{train}}} S_{k,j}^{(t+1)}. \tag{23}$$

Define the centered signal energy

$$E_{s,j}^{\mathrm{sig}} = (G_{s,j}^{(t)})^2 \sum_{k\in\mathcal{K}_{\mathrm{train}}} (S_{k,j}^{(t+1)} - \bar{S}_j^{(t+1)})^2, \tag{24}$$

and the intercept-sensitive noise energy

$$E_{s,j}^{\mathrm{int}} = |\mathcal{K}_{\mathrm{train}}|\,(\phi'(\eta_{s,j}^{(t)}))^2. \tag{25}$$

The local design-balance score is

$$d_{s,j} = \frac{\min\{E_{s,j}^{\mathrm{sig}}, E_{s,j}^{\mathrm{int}}\}}{\max\{E_{s,j}^{\mathrm{sig}}, E_{s,j}^{\mathrm{int}}\} + \varepsilon}. \tag{26}$$

The local residual-SNR score is

$$q_{s,j} = \frac{(G_{s,j}^{(t)})^2 v_j^{(t+1)}}{(G_{s,j}^{(t)})^2 v_j^{(t+1)} + \sigma_{s,j}^{2(t)} + \varepsilon}. \tag{27}$$

These are normalized by robust positive-entry medians:

$$\tau_d = \max(\text{median}\{d_{s,j} : d_{s,j} > 0\}, \varepsilon)\,, \tag{28}$$

$$\tau_q = \max(\text{median}\{q_{s,j} : q_{s,j} > 0\}, \varepsilon)\,, \tag{29}$$

$$\widetilde{d}_{s,j} = \text{clip}\left(\frac{d_{s,j}}{\tau_d}, 0, 1\right), \tag{30}$$

$$\widetilde{q}_{s,j} = \text{clip}\left(\frac{q_{s,j}}{\tau_q}, 0, 1\right). \tag{31}$$

The final sensor-specific information weight is

$$\omega_{s,j} = \text{clip}(\omega_j^{\text{freq}}\, \widetilde{d}_{s,j}\, \widetilde{q}_{s,j},\ \omega_{\min},\ 1). \tag{32}$$

The implementation also stores the conservative warning mask

$$m_{s,j} = m_j^{\text{freq}} \ \vee\ \mathbf{1}\{\omega_{s,j} \le \max(\omega_{\min}, 0.60)\}\,. \tag{33}$$

Step 5: one joint sensor update by sparse Gauss–Newton

Fix a sensor s and suppress the superscript (t) for readability. Let

$$g_j^{\text{ref}} = G_{s,j}^{(0)} \exp(\gamma_{s,j}), \tag{34}$$

$$n_j^{\text{ref}} = \phi(\eta_{s,j}), \tag{35}$$

$$h_j = \phi'(\eta_{s,j}), \tag{36}$$

$$x_{k,j} = g_j^{\text{ref}}\, S_{k,j}. \tag{37}$$

The nonlinear model is linearized around the current iterate:

$$Y_{s,k,j} \approx x_{k,j} + n_j^{\text{ref}} + x_{k,j}\, \delta\gamma_j + h_j\, \delta\eta_j. \tag{38}$$

The effective data weight for that sensor and frequency is

$$\widetilde{w}_{s,j} = w_{s,j}\, \omega_{s,j}. \tag{39}$$

Let $\delta\gamma, \delta\eta \in \mathbb{R}^J$ collect the per-frequency increments. Define

$$D_s^\gamma = \operatorname{diag}\left(\widetilde{w}_{s,j} \sum_k x_{k,j}^2\right)_{j=1}^J, \tag{40}$$

$$D_s^\eta = \operatorname{diag}\Big(\widetilde{w}_{s,j}\, |\mathcal{K}_{\text{train}}|\, h_j^2\Big)_{j=1}^J, \tag{41}$$

$$C_s = \operatorname{diag}\left(\widetilde{w}_{s,j}\, h_j \sum_k x_{k,j}\right)_{j=1}^J, \tag{42}$$

$$b_{s,j}^\gamma = \widetilde{w}_{s,j} \sum_k x_{k,j}(Y_{s,k,j} - x_{k,j} - n_j^{\text{ref}}), \tag{43}$$

$$b_{s,j}^\eta = \widetilde{w}_{s,j}\, h_j \sum_k (Y_{s,k,j} - x_{k,j} - n_j^{\text{ref}}). \tag{44}$$

Let $\lambda_{G,\text{ref}}$ and $\lambda_{N,\text{ref}}$ be the conservative reference penalties. Normally the reference targets are the current iterates:

$$\gamma_s^{\text{tar}} = \gamma_s, \qquad \eta_s^{\text{tar}} = \eta_s.$$

For the reliable sensor $s_\star$, the gain target is blended with zero:

$$\gamma_{s_\star}^{\text{tar}} = \frac{\lambda_{G,\text{ref}}\gamma_{s_\star} + \lambda_{\text{rel}}\, 0}{\lambda_{G,\text{ref}} + \lambda_{\text{rel}}}, \qquad \lambda_{G,\text{ref}}^{(\star)} = \lambda_{G,\text{ref}} + \lambda_{\text{rel}}. \tag{45}$$

For $s \neq s_\star$, one simply has $\lambda_{G,\text{ref}}^{(s)} = \lambda_{G,\text{ref}}$.

The update solves the sparse block system

$$\begin{bmatrix} D_s^\gamma + \lambda_{G,\text{smooth}} L + \lambda_{G,\text{ref}}^{(s)} I & C_s \\ C_s & D_s^\eta + \lambda_{N,\text{smooth}} L + \lambda_{N,\text{ref}} I \end{bmatrix} \begin{bmatrix} \delta\gamma_s \\ \delta\eta_s \end{bmatrix} = \begin{bmatrix} b_s^\gamma - \lambda_{G,\text{smooth}} L\gamma_s + \lambda_{G,\text{ref}}^{(s)}(\gamma_s^{\text{tar}} - \gamma_s) \\ b_s^\eta - \lambda_{N,\text{smooth}} L\eta_s + \lambda_{N,\text{ref}}(\eta_s^{\text{tar}} - \eta_s) \end{bmatrix}. \tag{46}$$

The raw update is then

$$\gamma_s^{\text{raw}} = \gamma_s + \delta\gamma_s, \qquad \eta_s^{\text{raw}} = \eta_s + \delta\eta_s. \tag{47}$$

If a correction cap $c_{\max} > 0$ dB is configured, the corresponding log-gain bound is

$$\gamma_{\max} = \log\Big(10^{c_{\max}/10}\Big), \tag{48}$$

and each raw gain correction is clipped to

$$-\gamma_{\max} \leq \gamma_{s,j}^{\text{raw}} \leq \gamma_{\max}. \tag{49}$$

Step 6: projection onto the identifiability constraint

The sensor updates are computed independently, so their raw log-gain corrections generally do not satisfy (7). The implementation therefore projects them frequency-by-frequency.

If no cap is active, then the projection is simply

$$\gamma_{s,j}^{(t+1)} = \gamma_{s,j}^{\text{raw}} - \frac{1}{M}\sum_{u=1}^{M}\gamma_{u,j}^{\text{raw}}. \tag{50}$$

If the cap is active, then for each j the code finds a scalar offset α_j by bisection such that

$$\frac{1}{M}\sum_{s=1}^{M}\text{clip}(\gamma_{s,j}^{\text{raw}} - \alpha_j, -\gamma_{\max}, \gamma_{\max}) = 0, \tag{51}$$

and sets

$$\gamma_{s,j}^{(t+1)} = \text{clip}(\gamma_{s,j}^{\text{raw}} - \alpha_j, -\gamma_{\max}, \gamma_{\max}). \tag{52}$$

The frequency-wise common log shift removed by this projection is

$$\Delta_j = \frac{1}{M}\sum_{s=1}^{M}(\gamma_{s,j}^{\text{raw}} - \gamma_{s,j}^{(t+1)}), \tag{53}$$

and is transferred into the latent spectra:

$$S_{k,j}^{(t+1)} \leftarrow S_{k,j}^{(t+1)}\exp(\Delta_j). \tag{54}$$

Finally,

$$G_{s,j}^{(t+1)} = G_{s,j}^{(0)}\exp(\gamma_{s,j}^{(t+1)}), \tag{55}$$

$$N_{s,j}^{(t+1)} = \phi(\eta_{s,j}^{\text{raw}}). \tag{56}$$

Step 7: residual-variance update

With the new parameters,

$$r_{s,k,j}^{(t+1)} = Y_{s,k,j} - G_{s,j}^{(t+1)}S_{k,j}^{(t+1)} - N_{s,j}^{(t+1)}, \tag{57}$$

and

$$\sigma_{s,j}^{2(t+1)} = \max\left(\frac{1}{|\mathcal{K}_{\text{train}}|}\sum_{k\in\mathcal{K}_{\text{train}}}\left(r_{s,k,j}^{(t+1)}\right)^2,\ \sigma_{\min}^2\right). \tag{58}$$

1.9 Monitored objective

The implementation stores, after each iteration, the diagnostic quantity

$$\mathcal{J}_{\text{diag}} = \frac{1}{2}\sum_{s=1}^{M}\sum_{j=1}^{J}\sum_{k\in\mathcal{K}_{\text{train}}}\left[\log\sigma_{s,j}^2 + \frac{r_{s,k,j}^2}{\sigma_{s,j}^2}\right] + \lambda_{G,\text{smooth}}\sum_{s=1}^{M}\|D_2\gamma_s\|_2^2 + \lambda_{N,\text{smooth}}\sum_{s=1}^{M}\|D_2\eta_s\|_2^2 + \lambda_{\text{rel}}\|\gamma_{s_\star}\|_2^2. \quad (59)$$

This is the quantity recorded in `objective_history`. It is a useful convergence diagnostic, but it is *not* the full time-varying function minimized by every inner linear solve, because the actual update equations also include iteration-dependent conservative reference terms and information weights.

1.10 Deployment-time calibration map

For any new sensor-first tensor of observations $Y_{s,\cdots,j}^{\text{new}}$, the deployed calibration map is

$$\widehat{S}_{s,\cdots,j}^{\text{new}} = \frac{Y_{s,\cdots,j}^{\text{new}} - N_{s,j}}{G_{s,j}}. \quad (60)$$

If nonnegativity is enforced, the implementation uses

$$\widehat{S}_{s,\cdots,j}^{\text{new}} = \max\left\{0,\ \frac{Y_{s,\cdots,j}^{\text{new}} - N_{s,j}}{G_{s,j}}\right\}. \quad (61)$$

The code broadcasts the curves $(G_{s,j}, N_{s,j})$ across any intermediate deployment dimensions while keeping the first axis as sensor and the last axis as frequency.

1.11 Consensus estimator

If several calibrated sensors later observe the same latent spectrum again, the implementation forms the inverse-variance consensus

$$\widehat{S}_{k,j}^{\text{net}} = \frac{\sum_{s=1}^{M}\sigma_{s,j}^{-2}\,\widehat{S}_{s,k,j}}{\sum_{s=1}^{M}\sigma_{s,j}^{-2}}. \quad (62)$$

This aggregation is meaningful only under a common-field assumption at deployment time.

1.12 Hold-out validation and returned diagnostics

Validation is performed by splitting whole calibration experiments into training and test subsets,

$$\mathcal{K}_{\text{train}} \cup \mathcal{K}_{\text{test}} = \{1, \ldots, K\}, \qquad \mathcal{K}_{\text{train}} \cap \mathcal{K}_{\text{test}} = \varnothing,$$

estimating the calibration on $\mathcal{K}_{\text{train}}$, and checking whether the deployed correction (60) reduces inter-sensor dispersion on $\mathcal{K}_{\text{test}}$.

In the current campaign adapter, hold-out reporting also includes per-sensor bias and RMSE to a leave-one-sensor-out corrected consensus, the fraction of bins where $Y_{s,k,j} \leq N_{s,j}$ would make the deployed correction invalid before nonnegativity truncation, the median information weight, the fraction of low-information bins, the fraction of bins that hit the gain cap, and the median alignment timing error.

The fitted result stores, in addition to (G, N, σ^2, S), the following diagnostics:

- the residual correction gain $\exp(\gamma_{s,j})$;
- the latent-variation curve v_j;
- the frequency-only weights ω_j^{freq} and mask m_j^{freq};
- the sensor-specific information weights $\omega_{s,j}$ and mask $m_{s,j}$;
- a mask of gain-correction bins that hit the configured cap;
- a mask of additive-noise bins numerically indistinguishable from zero.

1.13 Summary

The implemented estimator is therefore an alternating latent-variable scheme with the following specific structure:

1. load one campaign dataset, align rows by timestamp, and select a reliable sensor from ranking diagnostics;
2. set the nominal gain baseline to one;
3. represent gain by a residual log correction and noise by a softplus parameter;
4. update the latent spectra by weighted least squares;

5. detect weakly identifiable frequencies from latent variation and refine this into sensor-specific information weights;
6. update each sensor by one sparse, frequency-coupled Gauss–Newton solve with smoothness and conservative reference penalties;
7. project the raw gain corrections onto the mean-zero identifiability constraint, with an optional symmetric box constraint in dB;
8. update residual variances and repeat.

This is the actual solution implemented in the code. It is deliberately more conservative than the earlier idealized description: in weak-information bins it leans on regularization, damping, and projection rather than pretending that gain and additive floor are cleanly identifiable from the data.